Eksplorasi Informasi dan Struktur Citra Digital pada File JPEG

Diajukan untuk memenuhi Tugas Mata Kuliah Pengolahan Citra Digital

Dosen Pengampu : Andi Aljabar, M.Kom

Di Susun Oleh:

Nama : Dede Ahmad Fauzan

NIM : 24260014

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK DAN ILMU KOMPUTER

UNIVERSITAS NAHDLATUL ULAMA INDONESIA

2026

BAB I
PENDAHULUAN

1.1 Latar Belakang

Foto digital yang tampak sebagai gambar utuh bagi mata manusia sebenarnya adalah kumpulan angka yang tersusun dalam grid dua dimensi. Setiap angka mewakili intensitas cahaya pada satu titik, dan komputer hanya mengenali gambar melalui angka-angka tersebut. Di samping data piksel, sebuah file foto biasanya membawa metadata, yaitu keterangan tentang perangkat pengambil gambar, waktu pengambilan, orientasi, hingga lokasi. Seluruh informasi ini tersimpan dalam satu file dengan format tertentu, dan salah satu yang paling banyak digunakan adalah JPEG (Joint Photographic Experts Group).

Memahami isi dan struktur sebuah file citra merupakan dasar bagi pengolahan citra digital, visi komputer, forensik digital, dan analisis data visual. Tanpa pemahaman tersebut, operasi seperti penyesuaian kecerahan, deteksi tepi, kompresi, atau pengenalan objek hanya akan menjadi prosedur yang dijalankan tanpa dipahami maknanya. Eksplorasi sederhana terhadap satu foto nyata merupakan cara yang efektif untuk menjembatani konsep teoretis dengan data konkret.

Laporan ini menyajikan eksplorasi terhadap satu foto berformat JPEG bernama IMG_2.jpg, yang diambil menggunakan perangkat Xiaomi. Eksplorasi dilakukan melalui program Python berbasis library Pillow dan NumPy yang membaca informasi dasar file, metadata EXIF dan GPS, representasi piksel sebagai array angka, statistik warna, ukuran data mentah, hash file, serta karakteristik kompresi JPEG. Hasil keluaran program (Output.txt) menjadi sumber data seluruh pembahasan dalam laporan ini.

1.2 Rumusan Masalah

1. Informasi dasar dan metadata apa saja yang dapat dibaca dari file IMG_2.jpg?
2. Bagaimana sebuah foto digital direpresentasikan sebagai array angka, dan bagaimana struktur dimensinya?
3. Bagaimana karakteristik statistik warna, kecerahan, dan kontras foto tersebut?
4. Mengapa ukuran file JPEG jauh lebih kecil daripada ukuran data piksel mentahnya?
5. Bagaimana program Python bekerja untuk memperoleh seluruh informasi tersebut?

1.3 Tujuan

1. Mengidentifikasi informasi dasar file (nama, format, ukuran, resolusi, mode warna) dan metadata EXIF serta GPS.
2. Menjelaskan representasi foto sebagai array NumPy berdimensi tiga beserta makna setiap dimensinya.
3. Menghitung dan menafsirkan statistik kanal merah, hijau, dan biru, kecerahan, kontras, serta jumlah warna unik.
4. Membandingkan ukuran file terkompresi dengan ukuran data piksel mentah dan menjelaskan peran kompresi JPEG.
5. Menjelaskan identitas file melalui hash SHA-256 dan menguraikan alur kode Python yang digunakan.

BAB II
HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil eksplorasi terhadap file IMG_2.jpg secara berurutan, mulai dari alat yang digunakan, informasi dasar dan metadata, representasi piksel, statistik warna, ukuran dan struktur data, hingga identitas file. Bagian 2.8 menjelaskan kode Python yang menghasilkan seluruh data tersebut, dan bagian 2.9 merangkum pembahasan secara keseluruhan. Nomor dalam kurung siku, misalnya [9], merujuk pada nomor bagian pada keluaran program.

2.1 Alat dan Bahan

Eksplorasi dilakukan dengan satu skrip Python (analisis_foto.py) yang dijalankan pada file foto foto/IMG_2.jpg. Skrip tidak memerlukan koneksi internet ataupun perangkat khusus, cukup interpreter Python dan dua library eksternal.

Komponen	Jenis	Fungsi dalam eksplorasi
Python 3	Bahasa pemrograman	Menjalankan seluruh alur analisis
Pillow (PIL.Image, PIL.ExifTags)	Library pihak ketiga	Membuka gambar, membaca format, ukuran, mode warna,

Komponen	Jenis	Fungsi dalam eksplorasi
		metadata EXIF, GPS, dan tabel quantization
NumPy	Library pihak ketiga	Mengubah gambar menjadi array angka dan menghitung statistik (min, maks, mean, median, standar deviasi, nilai unik)
IMG_2.jpg	Bahan (data)	Foto JPEG yang dianalisis

Pembagian tugas kedua library utama bersifat saling melengkapi. Pillow berurusan dengan file dan metadata (apa yang tertulis di dalam berkas), sedangkan NumPy berurusan dengan isi piksel setelah gambar didekode menjadi matriks angka.

2.2 Data Citra yang Digunakan

Data yang dianalisis adalah satu foto berformat JPEG dengan nama IMG_2.jpg. Berdasarkan metadata, foto diambil dengan kamera perangkat Xiaomi bermodel 220333QAG pada 13 Agustus 2026 pukul 07:45:32. Foto berorientasi potret dengan resolusi 3.072 × 4.080 piksel.



Gambar 2.1. IMG_2.jpg

```
[1] INFORMASI DASAR FILE
------------------------------------------------------------------------
Nama file          : IMG_2.jpg
Ekstensi           : .jpg
Format gambar      : JPEG
Ukuran file        : 5,345,947 byte
Ukuran file        : 5220.65 KB
Ukuran file        : 5.10 MB
Lebar              : 3,072 pixel
Tinggi             : 4,080 pixel
Resolusi           : 3,072 x 4,080 pixel
Jumlah pixel       : 12,533,760 pixel
Mode warna         : RGB
```

Foto ini dipilih karena merupakan hasil kamera ponsel yang sesungguhnya, bukan gambar sintetis. Dengan demikian, metadata, kompresi, dan sebaran warna yang diperoleh mencerminkan kondisi file foto yang lazim dijumpai sehari-hari.

2.3 Hasil Analisis Informasi Dasar dan Metadata

Informasi dasar file

Program membaca nama dan ekstensi file dari objek Path, ukuran file dari sistem berkas, serta format, dimensi, dan mode warna dari objek gambar Pillow. Format yang dikenali Pillow adalah JPEG, sesuai dengan ekstensi .jpg. Ukuran file dilaporkan dalam tiga satuan (byte, KB, MB) agar mudah dibandingkan: 5.345.947 byte setara 5.220,65 KB atau 5,10 MB, dengan 1 KB = 1.024 byte dan 1 MB = 1.024 × 1.024 byte.

Lebar 3.072 piksel dan tinggi 4.080 piksel menghasilkan 12.533.760 piksel. Mode warna RGB berarti setiap piksel dijelaskan oleh tiga nilai: Red, Green, dan Blue.

Aspect ratio dan orientasi

Aspect ratio dihitung sebagai lebar dibagi tinggi, yaitu 3.072 ÷ 4.080 = 0,7529. Karena tinggi lebih besar daripada lebar, program menyimpulkan orientasi **portrait**. Jika disederhanakan dengan faktor persekutuan terbesar 48, rasio ini setara 64 : 85, yang sangat dekat dengan rasio foto 3 : 4 (0,75). Kedua dimensi habis dibagi 16 (3.072 = 16 × 192 dan 4.080 = 16 × 255), sehingga sesuai dengan blok pemrosesan JPEG yang dibahas pada bagian 2.7.

Metadata EXIF

EXIF (Exchangeable Image File Format) adalah blok metadata yang disisipkan kamera ke dalam file. Program melaporkan 12 tag EXIF pada tingkat dasar, lalu mencari 13 tag yang telah ditentukan di dalam kode. Hasilnya, 7 tag tersedia dan 6 tag dilaporkan tidak tersedia.

```
[3] METADATA / EXIF
--------------------------------------------------------------------------
Jumlah tag EXIF    : 12
Camera Make              : Xiaomi
Camera Model             : 220333QAG
Date/Time                : 2026:08:13 07:45:32
Date/Time Original       : Tidak tersedia
ISO                      : Tidak tersedia
Exposure Time            : Tidak tersedia
F-Number / Aperture      : Tidak tersedia
Focal Length             : Tidak tersedia
X Resolution             : 72.0
Y Resolution             : 72.0
Resolution Unit          : 2
Software                 : Tidak tersedia
Orientation              : 1
```

Catatan penting. Pada library Pillow, `getexif()` mengembalikan tag pada direktori utama (IFD0). Tag eksposur seperti ISO, waktu pencahayaan, bukaan, panjang fokus, dan Date/Time Original umumnya tersimpan di sub-direktori Exif terpisah, yang baru terbaca bila dipanggil secara eksplisit dengan `get_ifd(ExifTags.IFD.Exif)`. Oleh sebab itu, keterangan "Tidak tersedia" pada keluaran sebaiknya dibaca sebagai "tidak ditemukan oleh skrip ini", bukan sebagai bukti bahwa foto tidak memuat data tersebut. Ini adalah dugaan berdasarkan cara kerja library dan belum diverifikasi pada file ini.

Nilai resolusi 72 dpi hanyalah label untuk keperluan cetak dan tidak memengaruhi jumlah piksel. Pada 72 dpi, foto ini setara sekitar 42,7 × 56,7 inci, sedangkan pada 300 dpi (kualitas cetak umum) ukurannya sekitar 10,2 × 13,6 inci.

Informasi GPS

Program membaca direktori GPS dari EXIF dan mencetak setiap tag yang ditemukan. Foto ini memuat data GPS lengkap, sebagaimana ditunjukkan tabel berikut.

```
[4] INFORMASI GPS
----------------------------------------------------------------------
GPS                 : Tersedia
GPSLatitudeRef        : S
GPSLatitude           : (7.0, 27.0, 25.95)
GPSLongitudeRef       : E
GPSLongitude          : (107.0, 37.0, 22.17)
GPSAltitudeRef        : b'\x00'
GPSAltitude           : 0.0
GPSTimeStamp          : (0.0, 45.0, 16.0)
GPSProcessingMethod   : b'ASCII\x00\x00\x00CELLID\x00'
GPSDateStamp          : 2026:08:13
```

Koordinat dalam EXIF disimpan dalam format derajat, menit, dan detik. Konversi ke derajat desimal dilakukan dengan rumus berikut, dengan tanda negatif untuk lintang selatan dan bujur barat.

$$\text{derajat desimal} = D + \frac{M}{60} + \frac{S}{3600}$$

Untuk lintang: 7 + 27/60 + 25,95/3600 ≈ 7,4572, sehingga −7,4572° karena referensinya S. Untuk bujur: 107 + 37/60 + 22,17/3600 ≈ 107,6228°.

Ada tiga hal yang perlu dicermati dari data GPS ini:

1. **Akurasi rendah.** Nilai `GPSProcessingMethod` adalah CELLID, artinya posisi diperkirakan dari menara seluler, bukan dari satelit GPS. Akurasi metode ini umumnya jauh lebih kasar dibandingkan fix satelit, sehingga koordinat sebaiknya dipandang sebagai perkiraan area.

2. **Ketinggian tidak informatif.** Nilai 0,0 meter kemungkinan besar merupakan nilai bawaan karena metode CELLID tidak menyediakan data ketinggian. Ini adalah dugaan, bukan temuan terukur.
3. **Konsistensi waktu.** GPSTimeStamp 00:45:16 UTC bersesuaian dengan waktu EXIF 07:45:32 jika perangkat memakai zona UTC+7 (WIB): 07:45:32 dikurangi 7 jam adalah 00:45:32 UTC, hanya berselisih 16 detik dari cap waktu GPS. Kesesuaian ini mendukung keandalan metadata waktu, dengan asumsi foto diambil pada zona WIB.

Dari sisi privasi, koordinat tersebut berada di belahan bumi selatan dan timur, yaitu wilayah Pulau Jawa. Contoh ini menunjukkan bahwa foto yang dibagikan tanpa menghapus metadata dapat mengungkap lokasi pengambilan.

2.4 Hasil Analisis Representasi Piksel

Bagian ini menjawab pertanyaan inti eksplorasi: bagaimana sebuah foto menjadi angka. Perintah `np.array(image)` mendekode data JPEG yang terkompresi menjadi array NumPy, yaitu tabel angka yang dapat dihitung langsung.

Bentuk array

```
[5] REPRESENTASI GAMBAR SEBAGAI ANGKA
----------------------------------------------------------------------
Shape               : (4080, 3072, 3)
Jumlah dimensi      : 3
Tipe data           : uint8
Ukuran array        : 37,601,280 byte
Ukuran array        : 35.86 MB
```

Bentuk array tersusun sebagai tinggi × lebar × kanal, bukan lebar × tinggi seperti pada properti `image.size` milik Pillow. Perbedaan urutan ini penting. Dimensi pertama adalah posisi vertikal (baris, Y), dimensi kedua adalah posisi horizontal (kolom, X), dan dimensi ketiga adalah kanal warna: indeks 0 untuk Red, 1 untuk Green, dan 2 untuk Blue. Satu piksel diakses dengan `array_image[y, x]` dan menghasilkan tiga angka, misalnya [144, 160, 183].

```
array_image[y, x, c]
 y = 0 .. 4079 (baris, dari atas ke bawah)
```

x = 0 .. 3071 (kolom, dari kiri ke kanan)
c = 0, 1, 2 (Red, Green, Blue)

Program juga memverifikasi bahwa jumlah piksel hasil perkalian dimensi array (4.080 × 3.072 = 12.533.760) sama dengan jumlah piksel yang dibaca dari properti gambar [15]. Statusnya SESUAI, sehingga proses konversi dari gambar ke array tidak menghilangkan atau menambah piksel.

Contoh nilai piksel

Lima piksel dipilih secara manual untuk memperlihatkan bahwa setiap titik direpresentasikan oleh tiga angka.

Koordinat (x, y)	Nilai RGB	Kesan warna
(0, 0)	[144, 160, 183]	Biru keabuan terang, kanal Blue tertinggi
(10, 10)	[77, 93, 119]	Biru keabuan gelap
(100, 100)	[87, 95, 54]	Hijau zaitun, kanal Blue paling rendah
(500, 500)	[115, 125, 126]	Abu-abu hampir netral, ketiga kanal berdekatan
(1000, 500)	[96, 108, 70]	Hijau kekuningan

Piksel (0, 0) dan (10, 10) hanya berjarak sepuluh piksel secara diagonal, tetapi nilainya berbeda cukup jauh (kanal Red turun dari 144 ke 77). Hal ini menandakan area pojok kiri atas memiliki detail atau perubahan tonal yang cukup tajam. Nilai yang kanal-kanalnya hampir sama, seperti (500, 500), menunjukkan warna netral, sedangkan dominasi satu kanal menunjukkan warna yang condong ke kanal tersebut.

Sampling grid 5 × 5 [14]

Untuk melihat sebaran nilai pada seluruh bidang gambar, program mengambil 25 titik pada grid 5 × 5 menggunakan `np.linspace`. Titik grid jatuh pada x = 0, 767, 1535, 2303, 3071 dan y = 0, 1019, 2039, 3059, 4079 (hasil pembulatan ke bawah ke bilangan bulat).

y \ x	0	767	1535	2303	3071
0	[144, 160, 183]	[83, 103, 75]	[89, 103, 70]	[202, 211, 220]	[185, 192, 208]
1019	[192, 205, 213]	[211, 221, 230]	[222, 232, 242]	[209, 219, 229]	[191, 201, 210]
2039	[51, 61, 71]	[24, 32, 35]	[60, 71, 77]	[45, 54, 59]	[91, 95, 80]
3059	[119, 98, 71]	[35, 18, 8]	[21, 18, 13]	[11, 0, 0]	[64, 58, 26]
4079	[81, 64, 38]	[40, 29, 11]	[47, 45, 33]	[43, 45, 23]	[16, 24, 13]

Pola yang tampak dari grid, dibaca dari atas ke bawah:

- **Baris y = 0.** Campuran nilai hijau gelap di bagian tengah-kiri (misalnya [83, 103, 75]) dan nilai terang kebiruan di sisi kanan (misalnya [202, 211, 220]).
- **Baris y = 1019.** Seluruh titik terang dengan kanal Blue paling tinggi (hingga [222, 232, 242]), khas area langit, awan, atau permukaan terang yang agak kebiruan.
- **Baris y = 2039.** Seluruh titik jauh lebih gelap dengan kecenderungan biru keabuan, misalnya [24, 32, 35] dan [60, 71, 77].
- **Baris y = 3059 dan y = 4079.** Nilai sangat gelap, bernuansa cokelat atau hijau tua, dengan titik yang hampir hitam di (2303, 3059) yaitu [11, 0, 0].

Berdasarkan pola ini, foto kemungkinan memuat area atas yang terang (bisa jadi langit) dan area bawah yang gelap (bisa jadi tanah, bayangan, atau objek gelap) serta vegetasi di bagian atas-kiri. Tafsiran ini hanyalah dugaan dari 25 titik, karena foto itu sendiri tidak ditampilkan dalam laporan. Perubahan drastis kecerahan dari baris ke-2 ke baris ke-3 juga menjelaskan mengapa kontras foto tergolong tinggi (bagian 2.5).

2.5 Hasil Analisis Statistik Warna

Setelah gambar menjadi array, seluruh 12.533.760 piksel dapat diperlakukan sebagai data numerik. Ketiga kanal dipisahkan dengan pengirisan array (`array_image[:, :, 0]` untuk Red, `[:, :, 1]` untuk Green, `[:, :, 2]` untuk Blue), lalu masing-masing dihitung statistik deskriptifnya.

Rentang nilai dan statistik kanal [8][9][10]

Nilai piksel minimum untuk seluruh gambar adalah 0 dan maksimumnya 255. Rentang yang sama muncul pada masing-masing kanal.

Kanal	Minimum	Maksimum	Rata-rata	Median	Standar deviasi
Red	0	255	105,61	89,00	66,53
Green	0	255	106,65	91,00	71,38
Blue	0	255	96,56	68,00	81,80

Beberapa temuan dari tabel ini:

1. **Rentang penuh.** Setiap kanal memanfaatkan seluruh skala 8 bit dari 0 sampai 255. Keberadaan nilai 0 dan 255 menandakan adanya area yang sangat gelap dan area yang sangat terang. Jumlah piksel yang benar-benar "terpotong" (clipping) tidak dapat disimpulkan karena program tidak menghitung frekuensi nilai ekstrem.
2. **Rata-rata melebihi median pada semua kanal.** Selisihnya 16,61 (Red), 15,65 (Green), dan 28,56 (Blue). Pola ini menunjukkan distribusi miring ke kanan: sebagian besar piksel bernilai rendah (gelap), dengan ekor nilai tinggi dari area terang yang menarik rata-rata ke atas. Hal ini sejalan dengan temuan grid pada bagian 2.4, yaitu bagian bawah foto gelap dan bagian tengah-atas sangat terang.
3. **Kanal Blue paling beragam.** Standar deviasinya 81,80, tertinggi di antara tiga kanal, sementara median-nya 68 adalah yang terendah. Kombinasi median rendah dan sebaran lebar menunjukkan Blue bernilai sangat tinggi pada area terang kebiruan (misalnya [222, 232, 242]) dan sangat rendah pada area gelap atau kehijauan (misalnya [87, 95, 54]). Kemungkinan histogram Blue memiliki dua puncak, tetapi hal itu belum dapat dipastikan tanpa histogram.
4. **Tidak ada dominasi warna global yang kuat.** Rata-rata ketiga kanal berada pada rentang sempit, 96,56 sampai 106,65 (selisih sekitar 10 tingkat). Warna rata-rata foto sekitar RGB (106, 107, 97), yaitu abu-abu kehijauan yang sedikit hangat, karena kanal Blue sedikit lebih rendah.

Kecerahan (brightness) [11]

Kecerahan dihitung dengan rumus luminans yang memberi bobot berbeda pada tiap kanal sesuai sensitivitas mata manusia, yang paling peka terhadap hijau dan paling kurang peka terhadap biru.

Y = 0{,}299\,R + 0{,}587\,G + 0{,}114\,B

Ukuran kecerahan	Nilai
Minimum	0,00
Maksimum	255,00
Rata-rata	105,19
Median	88,83

Program mengelompokkan rata-rata kecerahan ke dalam empat kategori: di bawah 70 cenderung gelap, 70 sampai di bawah 140 sedang, 140 sampai di bawah 200 cenderung terang, dan 200 ke atas sangat terang. Dengan rata-rata 105,19 (sekitar 41% dari skala 255), foto dikategorikan **brightness sedang**.

Angka rata-rata ini dapat diperiksa dari statistik kanal: 0,299 × 105,61 + 0,587 × 106,65 + 0,114 × 96,56 = 105,19, persis sama dengan keluaran program. Pemeriksaan ini berlaku karena rata-rata bersifat linear, sedangkan median tidak. Median kecerahan (88,83) lebih rendah daripada rata-ratanya, konsisten dengan distribusi miring ke kanan: separuh piksel berada di bawah 89, yang termasuk gelap hingga sedang. Maka label "sedang" pada rata-rata sebaiknya dipahami sebagai hasil percampuran area sangat terang dan area gelap, bukan sebagai foto yang merata pada tingkat kecerahan menengah.

Kontras [12]

Kontras diukur sebagai standar deviasi kecerahan, yaitu seberapa jauh nilai kecerahan menyebar dari rata-ratanya. Hasilnya 70,47. Ambang yang dipakai program adalah di bawah 30 (rendah), 30 sampai di bawah 60 (sedang), dan 60 ke atas (tinggi), sehingga foto ini berkontras **relatif tinggi**.

Sebagai pembanding, data yang tersebar rata pada 0 sampai 255 memiliki standar deviasi sekitar 73,6 (255 dibagi akar 12), sedangkan batas teoretis tertinggi adalah 127,5 (separuh piksel hitam

dan separuh putih). Nilai 70,47 berarti kecerahan foto menyebar sangat lebar, mendekati sebaran yang rata, sehingga tidak ada satu rentang kecerahan yang menumpuk secara dominan. Ini sesuai dengan kontras tajam antara area atas yang terang dan area bawah yang gelap yang terlihat pada sampling grid.

Jumlah warna unik [13]

Program meratakan array menjadi 12.533.760 baris berisi tiga nilai RGB, lalu `np.unique(..., axis=0)` menghitung kombinasi yang berbeda. Hasilnya 192.992 warna unik.

Ukuran	Nilai
Jumlah piksel	12.533.760
Warna unik yang muncul	192.992
Warna yang mungkin pada RGB 8 bit (256^3)	16.777.216
Warna unik terhadap seluruh piksel	sekitar 1,54%
Warna unik terhadap seluruh kemungkinan warna	sekitar 1,15%
Rata-rata piksel per warna unik	sekitar 64,9

Foto hanya memakai sekitar 1,15% dari seluruh kemungkinan warna. Hal ini wajar untuk foto alami: warna berubah mulus dan saling berdekatan, sehingga nilai-nilai RGB cenderung mengelompok pada wilayah tertentu dalam ruang warna. Hasil ini juga menunjukkan bahwa banyak piksel berbagi kombinasi RGB yang sama, sebagaimana dinyatakan pada keluaran program. Angka "64,9 piksel per warna" hanyalah rata-rata, karena kenyataannya sebagian warna muncul ribuan kali dan sebagian lain hanya sekali.

2.6 Hasil Analisis Ukuran dan Struktur Data

Foto ini berukuran 5,10 MB sebagai file, tetapi membutuhkan 35,86 MB ketika dibuka sebagai array. Selisih tersebut adalah inti dari bagian ini.

Estimasi data piksel mentah [16]

Data mentah dihitung dari jumlah piksel dikalikan jumlah kanal dan ukuran tiap nilai:

$$\text{ukuran mentah} = \text{lebar} \times \text{tinggi} \times \text{kanal} \times \text{byte per nilai} = 3072 \times 4080 \times 3 \times 1$$

Besaran	Nilai
Byte per kanal (per nilai)	1
Jumlah kanal	3
Byte per piksel	3
Data piksel mentah	37.601.280 byte (35,86 MB)
Data mentah per kanal	12.533.760 byte (satu byte per piksel)

Hasil ini sama dengan ukuran array pada bagian [5] (37.601.280 byte), sehingga perhitungan teoretis dan ukuran array nyata saling mengonfirmasi.

Tipe data piksel [17]

Tipe data array adalah `uint8`, yaitu bilangan bulat tanpa tanda yang menempati 1 byte (8 bit). Delapan bit menyediakan $2^8 = 256$ tingkat intensitas, yaitu 0 sampai 255, dengan 0 sebagai intensitas terendah dan 255 tertinggi. Tiga kanal menghasilkan $256^3 = 16.777.216$ kombinasi warna yang mungkin, yang dikenal sebagai warna 24 bit (true color).

Pemilihan `uint8` juga berdampak pada memori. Perhitungan kecerahan pada bagian 2.5 menghasilkan bilangan pecahan sehingga disimpan sebagai `float64` (8 byte per nilai). Satu array kecerahan saja berukuran $12.533.760 \times 8 = 100.270.080$ byte atau sekitar 95,6 MB, lebih dari dua kali ukuran array gambar aslinya. Ini belum termasuk salinan sementara yang dibuat saat meratakan array dan mencari warna unik. Pada foto beresolusi tinggi, hal tersebut dapat menjadi perhatian bila memori terbatas.

Perbandingan dengan ukuran file JPEG

Ukuran file (5.345.947 byte) jauh lebih kecil daripada data mentah (37.601.280 byte). Program menjelaskan penyebabnya dalam catatan: JPEG menggunakan kompresi. Perbandingan kuantitatifnya adalah hasil perhitungan turunan dari dua angka tersebut.

Ukuran	Nilai

Ukuran	Nilai
Ukuran file JPEG	5.345.947 byte (5,10 MB)
Data piksel mentah	37.601.280 byte (35,86 MB)
Rasio kompresi	sekitar 7,03 : 1
Ukuran file terhadap data mentah	sekitar 14,22%
Penghematan ruang	sekitar 85,78%
Rata-rata bit per piksel pada file	sekitar 3,41 bit (data mentah: 24 bit)

Dengan rata-rata 3,41 bit per piksel, JPEG menyimpan setiap piksel menggunakan kurang dari seperenam dari 24 bit yang dibutuhkan data mentah. Kompresi JPEG bersifat lossy, yaitu membuang sebagian informasi yang dianggap kurang kasat mata. Oleh karena itu array hasil dekode adalah rekonstruksi dari data terkompresi, bukan salinan persis dari apa yang ditangkap sensor kamera. Ukuran file juga mencakup metadata (EXIF dan GPS) serta tabel pendukung kompresi, tetapi porsinya sangat kecil dibandingkan data gambar.

2.7 Identitas File dan Karakteristik JPEG

Hash SHA-256 [18]

Hash adalah "sidik jari digital": fungsi matematika yang mengubah isi file berukuran berapa pun menjadi deretan karakter dengan panjang tetap. Program menghitung SHA-256 dengan membaca file per potongan 4.096 byte, sehingga file besar tidak perlu dimuat seluruhnya ke memori. Hasilnya:

```
4c27f4877fec0e7637dadc9070421820c22a3319166284b67c9926f65e471236
```

Deretan tersebut terdiri atas 64 karakter heksadesimal, setara 256 bit. Sifat-sifat yang membuatnya berguna sebagai identitas file:

- **Deterministik.** File yang sama selalu menghasilkan hash yang sama, di komputer mana pun.
- **Sensitif.** Perubahan satu bit saja pada file menghasilkan hash yang sama sekali berbeda.
- **Satu arah.** File asli tidak dapat direkonstruksi dari hash.

- **Tahan tabrakan.** Secara praktis, tidak ada dua file berbeda yang menghasilkan hash sama.

Kegunaannya antara lain memverifikasi bahwa file tidak berubah selama pengiriman, mendeteksi duplikat, dan mencatat barang bukti dalam forensik digital. Perlu dicatat bahwa hash dihitung dari seluruh byte file, termasuk metadata. Dua foto yang tampak identik tetapi berbeda metadata (atau disimpan ulang dengan pengaturan kompresi berbeda) akan memiliki hash berbeda. Hash ini juga tidak membuktikan keaslian foto, hanya mengidentifikasi isi byte-nya.

Karakteristik JPEG [19]

Pillow mengenali file sebagai JPEG dan melaporkan adanya **2 tabel quantization**. Quantization adalah tahap kompresi yang membuang informasi. Dua tabel ini lazimnya berfungsi untuk komponen kecerahan (luminans) dan komponen warna (krominans), meskipun skrip tidak memeriksa peruntukan tiap tabel. Secara garis besar, JPEG bekerja dengan tahapan berikut:

1. **Konversi ruang warna.** RGB diubah menjadi YCbCr, yaitu satu komponen kecerahan (Y) dan dua komponen warna (Cb, Cr).
2. **Subsampling krominans (opsional).** Resolusi komponen warna sering dikurangi karena mata lebih peka terhadap kecerahan daripada warna.
3. **Pembagian blok 8 × 8.** Gambar dipecah menjadi blok kecil yang diproses terpisah.
4. **Transformasi DCT.** Setiap blok diubah dari nilai piksel menjadi koefisien frekuensi.
5. **Quantization.** Koefisien dibagi dengan nilai pada tabel quantization lalu dibulatkan. Tahap inilah sumber utama kehilangan informasi (lossy), terutama pada frekuensi tinggi yang kurang kasat mata.
6. **Pengodean entropi.** Hasilnya dipadatkan dengan pengodean Huffman.

Dimensi foto ini (3.072 × 4.080) habis dibagi 8 dan 16, sehingga gambar terbagi menjadi 384 × 510 = 195.840 blok 8 × 8 per kanal tanpa memerlukan penambahan piksel pengisi di tepi. Bila subsampling 4:2:0 dipakai, total blok yang diproses adalah 195.840 untuk Y ditambah 2 × 48.960 untuk Cb dan Cr, yakni 293.760 blok. Bila tanpa subsampling, totalnya 587.520 blok. Angka-angka ini merupakan perhitungan teoretis, karena skrip tidak membaca jenis subsampling maupun faktor kualitas file.

Dua keterbatasan perlu disebutkan. Pertama, jumlah tabel quantization tidak menunjukkan tingkat kualitas kompresi; untuk itu isi tabel harus diperiksa. Kedua, karena JPEG bersifat lossy, setiap kali foto disimpan ulang dengan JPEG, sebagian kualitas dapat berkurang lagi.

2.8 Penjelasan Kode Python

Skrip analisis_foto.py terdiri atas 607 baris yang disusun berurutan dari atas ke bawah tanpa fungsi atau kelas. Struktur ini sengaja dibuat sederhana agar tiap bagian dapat dibaca sebagai satu langkah pembelajaran. Skrip memiliki empat blok persiapan (import, penentuan lokasi file, header, pembukaan gambar) dan 20 bagian analisis bernomor [1] sampai [20] yang masing-masing mencetak satu judul pada keluaran.

Peta bagian program

Bagian	Isi	Library atau fungsi kunci
[1]	Informasi dasar file	Path.name, Path.suffix, os.path.getsize, image.size, image.format, image.mode
[2]	Aspect ratio dan orientasi	Pembagian width / height, percabangan if
[3]	Metadata EXIF	image.getexif(), kamus ID tag
[4]	Informasi GPS	exif.get_ifd(ExifTags.IFD.GPSInfo), ExifTags.GPSTAGS
[5]	Gambar menjadi array	np.array(image), .shape, .ndim, .dtype, .nbytes
[6]	Analisis dimensi array	Pembongkaran shape menjadi tinggi, lebar, kanal
[7]	Contoh nilai piksel	Pengindeksan array_image[y, x]
[8]	Nilai minimum dan	.min(), .max()

Bagian	Isi	Library atau fungsi kunci
	maksimum	
[9]	Statistik per kanal	Pengirisan kanal, .mean(), np.median, .std()
[10]	Rata-rata warna	.mean(axis=(0, 1))
[11]	Brightness	Rumus luminans, percabangan kategori
[12]	Kontras	brightness.std()
[13]	Warna unik	.reshape(-1, 3), np.unique(axis=0)
[14]	Sampling grid	np.linspace, perulangan bersarang
[15]	Verifikasi jumlah piksel	Perkalian dimensi, perbandingan
[16]	Estimasi data mentah	.itemsize, perkalian dimensi
[17]	Tipe data piksel	Perbandingan dtype == np.uint8
[18]	Hash file	hashlib.sha256, pembacaan per 4.096 byte
[19]	Informasi JPEG	image.quantization
[20]	Kesimpulan otomatis	f-string multibaris

Persiapan: import, lokasi file, dan pembukaan gambar

Tiga modul bawaan dan dua library eksternal diimpor di awal: Path (lokasi file), os (ukuran file), hashlib (hash), Image dan ExifTags dari Pillow, serta NumPy sebagai np. Lokasi file ditetapkan pada konstanta IMAGE_PATH = Path("foto/IMG_2.jpg"), yaitu jalur relatif terhadap folder

tempat skrip dijalankan. Bila file tidak ada, program berhenti segera dengan FileNotFoundError dan pesan yang jelas, sehingga kesalahan tidak muncul belakangan dalam bentuk pesan yang membingungkan.

Baris image = Image.open(IMAGE_PATH) membuat objek gambar. Pada tahap ini Pillow umumnya baru membaca header file (format, ukuran, mode), sedangkan dekode piksel penuh baru terjadi ketika data diminta, yaitu pada np.array(image). Itulah sebabnya bagian [1] sampai [4] dapat berjalan cepat dan bagian [5] menjadi tahap yang paling berat.

Bagian [1] dan [2]: informasi dasar dan aspect ratio

```
width, height = image.size
total_pixels = width * height
aspect_ratio = width / height
```

Properti image.size mengembalikan pasangan (lebar, tinggi). Jumlah piksel dihitung dengan perkalian, dan orientasi ditentukan dengan membandingkan lebar dan tinggi: lebih lebar berarti landscape, lebih tinggi berarti portrait, sama berarti square. Format angka {x:,} menambahkan pemisah ribuan sehingga bilangan besar mudah dibaca.

Bagian [3]: metadata EXIF

```
exif = image.getexif()
exif_tags = {271: "Camera Make", 272: "Camera Model", 306: "Date/Time", ...}
for tag_id, tag_name in exif_tags.items():
    value = exif.get(tag_id)
```

EXIF menyimpan tag sebagai nomor, bukan nama. Karena itu program membuat kamus yang memetakan nomor tag ke nama yang mudah dibaca (271 untuk merek, 272 untuk model, 306 untuk tanggal, dan seterusnya, 13 tag seluruhnya). Setiap tag dicari dengan exif.get(tag_id), yang mengembalikan None bila tidak ada, lalu dicetak "Tidak tersedia". Untuk tiga tag numerik (waktu eksposur, bukaan, panjang fokus), nilai diformat khusus menjadi detik, f/angka, dan mm di dalam blok try. Spesifikasi {tag_name:<25} meratakan nama ke kiri selama 25 karakter agar kolom tampak sejajar. Keterbatasan EXIF yang hanya dibaca pada direktori utama telah dibahas pada bagian 2.3.

Bagian [4]: GPS

```
gps_info = exif.get_ifd(ExifTags.IFD.GPSInfo)
for key, value in gps_info.items():
    tag_name = ExifTags.GPSTAGS.get(key, key)
```

Data GPS tersimpan pada direktori EXIF tersendiri. Fungsi get_ifd mengambil direktori tersebut, sedangkan ExifTags.GPSTAGS menerjemahkan nomor tag GPS menjadi nama seperti GPSLatitude. Pemanggilan dibungkus try ... except agar program tetap berjalan bila direktori GPS tidak ada. Perhatikan bahwa program mencetak nilai mentah dan tidak mengonversi koordinat menjadi derajat desimal, sehingga konversi pada bagian 2.3 dilakukan secara manual.

Bagian [5] sampai [7]: gambar menjadi angka

```
array_image = np.array(image)
array_height, array_width, channels = array_image.shape
pixel = array_image[y, x]
```

np.array(image) adalah titik balik program: gambar berubah dari objek berkas menjadi matriks angka. Atribut .shape, .ndim, .dtype, dan .nbytes menjelaskan bentuk, jumlah dimensi, tipe data, dan ukuran memori. Bagian [6] membongkar shape menjadi tiga variabel (tinggi, lebar, kanal) dan mencetak makna tiap dimensi, dengan cabang elif array_image.ndim == 2 untuk menangani gambar grayscale. Bagian [7] mengambil piksel pada daftar koordinat (x, y) tetapi mengindeks array sebagai array_image[y, x], karena NumPy menyimpan baris sebelum kolom. Pemeriksaan 0 <= x < width and 0 <= y < height mencegah kesalahan indeks di luar gambar.

Bagian [8] sampai [12]: statistik, kecerahan, dan kontras

```
red = array_image[:, :, 0]
mean_rgb = array_image[:, :, :3].mean(axis=(0, 1))
brightness = 0.299 * red + 0.587 * green + 0.114 * blue
contrast = brightness.std()
```

Pengirisan [:, :, 0] mengambil seluruh baris dan kolom pada kanal 0 saja, sehingga menghasilkan matriks dua dimensi untuk satu warna. Fungsi .min(), .max(), .mean(), np.median(), dan .std() menghitung statistik atas seluruh isi matriks tersebut. Argumen axis=(0, 1) pada bagian [10] meminta rata-rata dihitung sepanjang baris dan kolom sekaligus, menyisakan satu nilai per kanal.

Operasi `0.299 * red` mengalikan seluruh matriks dengan bilangan pecahan sehingga hasilnya berubah menjadi `float64`, dan inilah yang membuat nilai kecerahan dapat berupa pecahan seperti 105,19. Standar deviasi pada NumPy secara bawaan menghitung simpangan populasi (pembagi N), yang wajar karena seluruh piksel foto dihitung, bukan sampel. Kategori kecerahan dan kontras ditentukan dengan rangkaian `if ... elif ... else` terhadap ambang yang telah dibahas pada bagian 2.5.

Bagian [13]: warna unik

```
pixels = array_image[:, :, :3].reshape(-1, 3)
unique_colors = np.unique(pixels, axis=0)
```

`reshape(-1, 3)` meratakan matriks tinggi × lebar × 3 menjadi daftar 12.533.760 baris berisi tiga nilai, dengan `-1` berarti "hitung jumlah baris secara otomatis". `np.unique(..., axis=0)` kemudian menyortir baris-baris itu dan membuang duplikat. Operasi ini adalah salah satu yang paling berat dalam skrip, karena melibatkan pengurutan jutaan baris.

Bagian [14]: sampling grid

```
grid_x = np.linspace(0, width - 1, 5, dtype=int)
grid_y = np.linspace(0, height - 1, 5, dtype=int)
```

`np.linspace` menghasilkan lima nilai berjarak sama dari 0 sampai indeks terakhir. Argumen `dtype=int` memotong desimal ke bawah (misalnya 1019,75 menjadi 1019). Dua perulangan bersarang menelusuri setiap kombinasi (x, y) dan mencetak pikselnya, sehingga diperoleh 25 titik yang tersebar merata.

Bagian [15] sampai [17]: verifikasi dan ukuran data

Bagian [15] membandingkan hasil kali dimensi array dengan `total_pixels` dari Pillow sebagai uji konsistensi. Bagian [16] menghitung ukuran mentah dengan rumus lebar × tinggi × kanal × `itemsize` dan mencetak catatan bahwa file JPEG lebih kecil karena kompresi. Bagian [17] memeriksa `array_image.dtype == np.uint8` lalu menjelaskan rentang 0 sampai 255.

Bagian [18]: hash file

```
sha256 = hashlib.sha256()
with open(IMAGE_PATH, "rb") as file:
```

```
    while True:
        data = file.read(4096)
        if not data:
            break
        sha256.update(data)
```

File dibuka dalam mode biner ("rb") dan dibaca bertahap 4.096 byte. Setiap potongan dimasukkan ke objek hash melalui update(), dan hexdigest() menghasilkan hasil akhir dalam heksadesimal. Cara bertahap ini hemat memori dan menghasilkan hash yang sama dengan membaca seluruh file sekaligus. Penggunaan with memastikan file ditutup otomatis.

Bagian [19] dan [20]: informasi JPEG dan kesimpulan

Bagian [19] memastikan image.format == "JPEG" lalu membaca image.quantization, yaitu kamus berisi tabel quantization, dan mencetak jumlahnya. Bagian [20] mencetak ringkasan melalui f-string multibaris, di mana nilai seperti nama file, resolusi, jumlah piksel, dan rentang nilai disisipkan otomatis dari variabel. Dengan demikian, kesimpulan akan menyesuaikan diri bila skrip dijalankan pada foto lain. Meski demikian, sebagian butir kesimpulan (misalnya nomor 7 sampai 11) berupa kalimat tetap yang tidak bergantung pada hasil.

Ketergantungan antarbagian dan potensi perbaikan

Beberapa bagian bergantung pada variabel yang dibuat di bagian sebelumnya: channels dari bagian [6] dipakai di [9] dan [16], red, green, blue dari [9] dipakai di [11], dan brightness dari [11] dipakai di [12]. Skrip karenanya harus dijalankan berurutan. Pada foto grayscale (dua dimensi), variabel tersebut tidak terbentuk sehingga bagian [11] dan [12] tidak akan berjalan sebagaimana mestinya.

Beberapa perbaikan dapat dipertimbangkan, dan semuanya merupakan saran, bukan cacat yang mengubah hasil pada foto ini:

1. Membaca tag eksposur dari sub-direktori Exif dengan exif.get_ifd(ExifTags.IFD.Exif) agar ISO, waktu eksposur, bukaan, dan panjang fokus dapat ditampilkan bila ada.
2. Mengonversi koordinat GPS menjadi derajat desimal di dalam kode.
3. Mengganti except: kosong dengan except (TypeError, ValueError): agar hanya galat yang diharapkan yang ditangkap.

4. Memanggil `image.convert("RGB")` sebelum dikonversi ke array agar mode lain (misalnya RGBA atau P) ditangani seragam.
5. Membuat histogram kanal untuk memastikan dugaan distribusi miring dan dua puncak pada bagian 2.5.

2.9 Pembahasan Keseluruhan

File `IMG_2.jpg` dapat dipahami sebagai paket berlapis: wadah file yang dikompresi, metadata yang menyertainya, dan data piksel yang tersembunyi di dalam kompresi tersebut. Eksplorasi ini membuka ketiga lapisan itu satu per satu.

Ringkasan temuan

Aspek	Temuan utama	Makna
File	JPEG, 5.345.947 byte (5,10 MB), 3.072 × 4.080 piksel, RGB	Foto potret beresolusi sekitar 12,53 megapiksel
Metadata	Xiaomi 220333QAG, 13 Agustus 2026 07:45:32, orientasi normal	Foto berasal dari kamera ponsel dan tidak perlu diputar
GPS	−7,4572°, 107,6228°; metode CELLID; ketinggian 0,0	Lokasi tersimpan dan berakurasi kasar; berisiko bagi privasi
Representasi	Array (4080, 3072, 3) bertipe uint8, 35,86 MB	Foto sepenuhnya berupa 37.601.280 angka 0 sampai 255
Statistik warna	Rata-rata RGB sekitar (106, 107, 97); median lebih rendah dari rata-rata	Warna rata-rata netral, distribusi condong ke piksel gelap
Kecerahan dan kontras	Rata-rata 105,19 (sedang);	Area sangat terang dan sangat

Aspek	Temuan utama	Makna
	standar deviasi 70,47 (tinggi)	gelap hadir bersamaan
Warna unik	192.992 dari 16.777.216 kemungkinan (sekitar 1,15%)	Foto alami memakai sebagian kecil ruang warna
Kompresi	Rasio sekitar 7,03 : 1; sekitar 3,41 bit per piksel	JPEG menghemat sekitar 85,78% ruang
Identitas	SHA-256 4c27f487…471236	Sidik jari byte file, bukan jaminan keaslian

Keterkaitan antartemuan

Temuan-temuan di atas saling menguatkan. Sampling grid memperlihatkan baris terang di sekitar y = 1019 dan baris-baris gelap di bawahnya. Pola itu menjelaskan mengapa rata-rata kecerahan (105,19) berada di atas median (88,83), mengapa standar deviasi kecerahan mencapai 70,47, dan mengapa rata-rata Blue (96,56) jauh di atas median-nya (68). Nilai 0 dan 255 yang sama-sama muncul pada setiap kanal juga cocok dengan adanya titik hampir hitam seperti [11, 0, 0] dan titik sangat terang seperti [222, 232, 242].

Di sisi lain, konsistensi internal metadata dan perhitungan memberi dasar kepercayaan terhadap hasil. Waktu GPS (00:45:16 UTC) berselisih hanya 16 detik dari waktu EXIF bila dikonversi dari WIB, rata-rata kecerahan cocok dengan rata-rata kanal berbobot, ukuran array teoretis sama dengan ukuran array terukur, dan jumlah piksel pada Pillow sama dengan hasil kali dimensi array.

Tiga bentuk satu foto

Eksplorasi ini menunjukkan tiga bentuk representasi dari foto yang sama, masing-masing dengan kegunaan berbeda:

1. **File terkompresi (5,10 MB).** Bentuk untuk penyimpanan dan pengiriman, efisien tetapi tidak dapat dihitung secara langsung.
2. **Array piksel (35,86 MB).** Bentuk untuk pemrosesan, karena setiap piksel dapat diakses dan dihitung, namun boros memori.

3. **Ringkasan statistik (beberapa belas angka).** Bentuk untuk penafsiran cepat, tetapi kehilangan informasi spasial. Rata-rata dan standar deviasi tidak memberi tahu di bagian mana foto itu terang atau gelap.

Perpindahan antarbentuk selalu melibatkan pertukaran antara ukuran, kemudahan pengolahan, dan kelengkapan informasi.

Isu privasi

Foto ini membawa koordinat lokasi, model perangkat, dan waktu pengambilan tanpa terlihat di gambar. Meskipun metode CELLID hanya memberi perkiraan area, kombinasi lokasi, tanggal, dan waktu tetap dapat mengungkap kebiasaan atau keberadaan seseorang. Sebelum foto dibagikan secara publik, metadata sebaiknya dihapus atau fitur penyimpanan lokasi pada kamera dinonaktifkan. Sebagian platform media sosial menghapus metadata saat mengunggah, tetapi perilaku ini berbeda-beda sehingga tidak boleh diandalkan.

Keterbatasan

1. **Satu sampel.** Seluruh temuan berasal dari satu foto. Foto lain dengan pencahayaan, kamera, atau pengaturan kompresi berbeda akan menghasilkan angka berbeda.
2. **Pembacaan EXIF tidak lengkap.** Enam tag dilaporkan "Tidak tersedia" dan kemungkinan sebagian besar berada di sub-direktori yang tidak dibaca skrip (lihat bagian 2.3 dan 2.8). Kesimpulan tentang ketiadaan ISO, eksposur, bukaan, dan panjang fokus tidak boleh ditarik dari keluaran ini.
3. **Tafsiran isi gambar bersifat dugaan.** Dugaan tentang langit, vegetasi, atau area gelap hanya bersandar pada 25 titik grid dan sejumlah piksel sampel. Foto tidak ditampilkan dan tidak dianalisis secara visual.
4. **Kecerahan sederhana.** Rumus 0,299 R + 0,587 G + 0,114 B diterapkan langsung pada nilai piksel yang telah dikodekan gamma, sehingga merupakan pendekatan kecerahan persepsi (luma) dan bukan pengukuran luminans fisik yang presisi. Ambang kategori (70, 140, 200 untuk kecerahan; 30 dan 60 untuk kontras) adalah kesepakatan dalam program, bukan standar baku.

5. **Statistik tanpa distribusi.** Rata-rata, median, dan standar deviasi belum menggambarkan bentuk distribusi. Dugaan distribusi miring dan dua puncak pada Blue memerlukan histogram untuk dipastikan.
6. **Parameter kompresi tidak diperiksa.** Skrip hanya menghitung tabel quantization, tanpa membaca isi tabel, jenis subsampling, atau faktor kualitas. Rasio kompresi 7,03 : 1 adalah hasil hitung dari dua ukuran, bukan pengukuran langsung pengaturan kamera.
7. **Ketinggian GPS 0,0 tidak dapat diandalkan** dan akurasi posisi CELLID rendah.

Keterbatasan tersebut tidak mengurangi nilai eksplorasi, karena tujuannya adalah memahami struktur citra digital. Namun, keterbatasan ini menjadi batas yang perlu diingat saat menafsirkan angka.

BAB III
PENUTUP

3.1 Kesimpulan

Eksplorasi terhadap file IMG_2.jpg menunjukkan bahwa sebuah foto digital adalah gabungan data piksel numerik dan metadata dalam wadah terkompresi, dan seluruhnya dapat dibaca serta dianalisis dengan Python. Kesimpulan berikut menjawab rumusan masalah pada Bab I.

1. **Informasi dasar dan metadata.** File berformat JPEG berukuran 5.345.947 byte (5,10 MB) dengan resolusi 3.072 × 4.080 piksel (12.533.760 piksel), mode warna RGB, aspect ratio 0,7529 (portrait, mendekati 3 : 4). Metadata EXIF mencatat perangkat Xiaomi 220333QAG, waktu pengambilan 13 Agustus 2026 pukul 07:45:32, dan orientasi normal. Data GPS tersedia (sekitar 7,4572° LS dan 107,6228° BT) dengan metode CELLID sehingga akurasinya kasar. Tag eksposur yang dilaporkan tidak tersedia kemungkinan besar tidak terbaca oleh skrip, bukan benar-benar tidak ada.
2. **Representasi sebagai angka.** Foto direpresentasikan sebagai array NumPy berbentuk (4080, 3072, 3) bertipe uint8, yaitu 37.601.280 nilai bulat berkisar 0 sampai 255 yang disusun sebagai baris (Y), kolom (X), dan kanal (R, G, B). Setiap piksel dijelaskan oleh tiga angka, dan jumlah piksel hasil perhitungan array sesuai dengan jumlah piksel pada gambar.
3. **Karakteristik warna.** Seluruh kanal menjangkau rentang penuh 0 sampai 255 dengan rata-rata RGB sekitar (105,61; 106,65; 96,56), yaitu warna rata-rata netral kehijauan.

Rata-rata kanal selalu lebih besar daripada median, yang menandakan dominasi piksel gelap disertai area terang. Kecerahan rata-rata 105,19 tergolong sedang, kontras (standar deviasi 70,47) tergolong tinggi, dan terdapat 192.992 warna unik atau sekitar 1,15% dari 16.777.216 kemungkinan warna.

4. **Ukuran dan kompresi.** Data piksel mentah berukuran 37.601.280 byte (35,86 MB), sekitar 7,03 kali lebih besar daripada file JPEG. Selisih ini terjadi karena JPEG memakai kompresi lossy berbasis blok 8 × 8, DCT, quantization (2 tabel pada file ini), dan pengodean entropi, yang menyimpan rata-rata sekitar 3,41 bit per piksel dibandingkan 24 bit pada data mentah.
5. **Identitas file dan program.** Hash SHA-256 `4c27f4877fec0e7637dadc9070421820c22a3319166284b67c9926f65e471236` mengidentifikasi isi byte file secara unik. Program Python mencapai seluruh hasil tersebut melalui alur terurut: Pillow membaca file dan metadata, NumPy mengubah gambar menjadi array dan menghitung statistik, `hashlib` menghitung hash, dan f-string menyusun kesimpulan otomatis.

Secara keseluruhan, tujuan laporan tercapai: struktur informasi pada file JPEG dapat diuraikan dari tingkat berkas hingga tingkat piksel. Hasil ini berlaku untuk satu foto dan memiliki keterbatasan yang telah diuraikan pada bagian 2.9, terutama pembacaan EXIF yang belum lengkap, belum adanya histogram, dan tafsiran isi gambar yang hanya bersifat dugaan. Eksplorasi ini dapat dilanjutkan dengan pembacaan sub-direktori Exif, pembuatan histogram, analisis beberapa foto sekaligus, atau penerapan pengolahan citra seperti penyesuaian kecerahan, penajaman, dan deteksi tepi.